Лабораторная работа № 3 — Многоэкранная навигация, формы и валидация

Дисциплина	Программирование мобильных приложений (PAM)
Период	недели 6–7
Защита	индивидуальная, на занятиях
Автор	унив. лектор Ionel Antohi

1. Цели работы

По завершении работы студент способен:

1. установить пакет из экосистемы pub.dev командой `flutter pub add` и объяснить, что изменилось в `pubspec.yaml`;
2. описать **топологию приложения** в виде карты маршрутов `go_router` (`lib/router.dart`), подключённой через `MaterialApp.router`;
3. построить цепочку **список** → **детали** с параметром пути (`/:id`) и осознанно выбирать между `context.push` и `context.go`;
4. обернуть главные пункты назначения темы в **постоянные вкладки** (`StatefulShellRoute` + `NavigationBar`);
5. превратить формы-макеты из L2 в формы, **функциональные локально**: `Form`, `TextFormField`, `validator`, контроллеры, освобождаемые в `dispose`, подходящие клавиатуры и фильтры;
6. локально валидировать экран аутентификации и убирать login из стека после успешного входа;
7. давать feedback: `SnackBar` при действиях, диалог подтверждения при деструктивных действиях.

Чего эта работа НЕ делает: разделяемое состояние — корзина, которая помнит, список, который действительно растёт после submit, — это **материал L4**; реальная аутентификация приходит на **L5**, с backend-ом. Контракт L3: приложение **навигирует по адресам**, формы **отказывают плохим данным и подтверждают хорошие** — но данные остаются mock. Не устанавливайте Provider или другие библиотеки состояния — форма справляется сама, как показано на C6.

2. Предварительные условия

- **L2 защищена**: все обязательные экраны из ТЗ существуют как статический UI, с переиспользуемыми widget-ами и инкрементальной историей Git.
- **Лекция C6 пройдена**: callback-и, `SnackBar` и диалоги, `Form` + `TextFormField` + `validator`, `go_router` (карта, `go` vs `push`, `/:id`). Работа применяет ровно материал C6 к вашему проекту и добавляет **`ShellRoute` шаг за шагом**, обещанный на лекции.

- **Проект L2 работоспособен**: `flutter run` запускает приложение без ошибок.
- Держите открытым **пункт L3 вашей темы** из каталога: он называет цепочку навигации и валидации, которые будут искать критерии оценивания (телефон у MediShop, числовая длительность у FitTrack, deadline в будущем у EduPlan, положительная сумма у WalletMate…).

3. Задание 1 — Непрерывность: тот же проект, первые два commit-а

НЕ создавайте новый проект. L3 растёт **прямо на проекте из L2** — на защите `git log` показывает одно и то же приложение, растущее с недели 4 до сегодня. Кто начинает «с чистого листа», теряет баллы за непрерывность.

Первый commit L3 — установка `go_router`, первого внешнего пакета проекта:

```
flutter pub add go_router
git add .
git commit -m "L3: добавлен пакет go_router"
```

Команда записывает зависимость в `pubspec.yaml`; точную версию пинует `pubspec.lock` — ритуал pub.dev с C6. Второй commit: карта маршрутов (Задание 2). **Дисциплина Git из L2 остаётся**: `git add .` + commit после каждого завершённого шага, с сообщениями, которые говорят, что было добавлено.

4. Задание 2 — Карта маршрутов: `router.dart` и `MaterialApp.router`

4.1. Модели обретают идентичность: поле `id`

Адрес `/records/7` требует, чтобы пластинка № 7 **существовала** — mock-модель из L2 не имела `id`. Добавьте его, ровно как в демо C6 (если модель вашей темы уже имеет `id`, пропустите шаг):

```dart
// data/mock_data.dart — модель обретает идентичность
class VinylRecord {
  final int id; // НОВОЕ: идентичность, которая будет путешествовать в адресе
  final String title;
  // ... остальные поля из L2; id идёт первым в конструкторе
}
// в mockRecords — уникальные id: VinylRecord(1, 'Kind of Blue', ...), ...
```

4.2. Карта: все экраны, с адресами

Создайте `lib/router.dart` со **всеми обязательными экранами из ТЗ** как маршрутами — каждый получает адрес; думайте об адресах как о маленьком REST API (параллель станет буквальной на L5):

```dart
// lib/router.dart — ВСЕ экраны, с адресами (import-ы — сверху)
final router = GoRouter(
  initialLocation: '/login', // приложение стартует в аутентификации
  routes: [
    GoRoute(path: '/login',
        builder: (context, state) => const LoginScreen()),
    GoRoute(path: '/records',
        builder: (context, state) => const CollectionScreen()),
    GoRoute(
      path: '/records/:id', // переменный сегмент: /records/7, /records/42
      builder: (context, state) {
        final id = int.parse(state.pathParameters['id']!);
        return RecordDetailScreen(id: id);
      },
    ),
    // ... так же: '/records/new' (ФИКСИРОВАННЫЙ — перед ':id'!), '/wishlist',
    //     '/profile' — ВСЕ экраны ТЗ получают маршрут
  ],
);
```

Две ловушки, прямо из практики защит: **порядок маршрутов имеет значение** — они пробуются сверху вниз, поэтому `/records/new` стоит **перед** `/records/:id`, иначе «new» ловится переменным сегментом и `int.parse('new')` падает; и **параметр пути — всегда `String`** — адрес это текст, конверсия (`int.parse`) — ваша; знак `!` оправдан: карта гарантирует, что на этом маршруте `:id` существует.

4.3. Подключение: `MaterialApp.router` заменяет `home:`

В `main.dart` `MaterialApp` становится `MaterialApp.router` — параметр `home:` из L2 **исчезает** (кто ставит оба, получает ошибку; стартовый экран теперь `initialLocation`). Визуальная тема остаётся нетронутой:

```dart
// main.dart — навигацию принимает router; home: исчезает
return MaterialApp.router(
  title: 'Vinyl Shelf',
  theme: ThemeData(/* ваша тема из L2, без изменений */),
  routerConfig: router,
);
```

Правило из демо C6: `router` — глобальный объект, построенный один раз — **изменения карты требуют restart (R), а не hot reload (r)**. Удалите теперь и временные строительные леса из L2 (тамошнее Задание 6): кнопки с `Navigator.push` + `MaterialPageRoute` исчезают — с этого commit-а вся навигация идёт по адресам.

5. Задание 3 — Цепочка темы: список → детали с `/:id`

Постройте цепочку навигации, названную в **пункте L3 вашей темы** (каталог → детали → корзина, расписание → детали курса → задания, панель → транзакции → форма…). Шаблон, на типовом примере:

Шаг 1 — карточка оживает. Переиспользуемый widget из L2 получает параметр `onTap` (`final VoidCallback? onTap;`, передаваемый `ListTile`-у — пустое место, намеренно оставленное на L2, Задание 5), а список отправляет к деталям с id, интерполированным в адрес:

```
// screens/collection_screen.dart — в itemBuilder-е списка из L2
RecordCard(
  record: record,
  onTap: () => context.push('/records/${record.id}'), // зайду и вернусь
),
```

Шаг 2 — детали сами ищут свой объект. В адресе путешествует **идентичность**: экран получает id через конструктор и ищет сам — сегодня в mock-е, на L4 через Repository, на L5 из backend-а; адрес остаётся, источник данных меняется под низом:

```
// screens/record_detail_screen.dart — экран ищет по id
final record = mockRecords.firstWhere((r) => r.id == id);
// AppBar(title: Text(record.title)) — стрелка назад появляется сама
```

Шаг 3 — `push` или `go`, на каждом прыжке. Правило с C6, применённое к проекту: **`push`** для «зайду и вернусь» — список → детали, открытие формы (закрываемой затем через `context.pop()`); **`go`** для прыжков между разделами — между вкладками и, обязательно, **после login** (Задание 6): стек становится стеком адреса, дорога назад исчезает.

Ловушка `extra`: go_router может нести рядом с адресом целый объект (`extra:`) — не делайте этого: `extra` не входит в адрес, а значит умирает ровно в тех сценариях, ради которых мы приняли маршрутизацию по адресам (прямая ссылка, восстановление состояния, refresh в вебе). Id в пути + поиск в экране — идиома, которую отмечают критерии оценивания.

6. Задание 4 — Постоянные вкладки: `StatefulShellRoute` шаг за шагом

Обещание, данное на C6, выполняется здесь: главные пункты назначения стоят под `NavigationBar`-ом, который **остаётся на экране**, пока содержимое меняется. Используем `StatefulShellRoute.indexedStack`: каждая вкладка сохраняет собственный стек и состояние (scroll остаётся там, где вы его оставили), а связка с `NavigationBar` пишется в две строки — без ручного сопоставления адрес → индекс, которого потребовал бы простой `ShellRoute` из документации пакета.

Шаг 1 — выберите пункты назначения. Из ТЗ темы: **3–5 экранов верхнего уровня** — главный список, раздел «мои» (заказы, взятые книги, билеты…), профиль — верный кандидат. **Во вкладки НЕ входят**: login (остаётся вне shell-а) и экраны глубины — детали и формы становятся **под-маршрутами** своей вкладки.

Шаг 2 — напишите обёртку. Новый widget со `Scaffold`-ом, который держит панель; `StatelessWidget` — текущий индекс знает router, поэтому леса `StatefulWidget` из L2 (у тех, кто выбрал вариант с `NavigationBar`) удаляются:

```dart
// widgets/app_shell.dart — обёртка с постоянными вкладками
class AppShell extends StatelessWidget {
  final StatefulNavigationShell navigationShell;
  const AppShell({super.key, required this.navigationShell});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: navigationShell, // содержимое текущей вкладки
      bottomNavigationBar: NavigationBar(
        selectedIndex: navigationShell.currentIndex,
        onDestinationSelected: navigationShell.goBranch, // переключает вкладку
        destinations: const [
          NavigationDestination(icon: Icon(Icons.album), label: 'Коллекция'),
          NavigationDestination(icon: Icon(Icons.favorite), label: 'Избранное'),
          NavigationDestination(icon: Icon(Icons.person), label: 'Профиль'),
        ],
      ),
    );
  }
}
```

Шаг 3 — перестройте карту, в два хода. Сначала детали и форма спускаются как **дочерние маршруты** под `/records` — пути дочерних маршрутов **относительные** (без `/` в начале), а правило порядка из Задания 2 сохраняется:

```
// маршрут '/records' — теперь со своими под-маршрутами
GoRoute(
  path: '/records',
  builder: (context, state) => const CollectionScreen(),
  routes: [
    GoRoute(
      path: 'new', // полный адрес: /records/new — по-прежнему перед ':id'
      builder: (context, state) => const RecordFormScreen(),
    ),
    GoRoute(
      path: ':id', // полный адрес: /records/:id
      builder: (context, state) {
        final id = int.parse(state.pathParameters['id']!);
        return RecordDetailScreen(id: id);
      },
    ),
  ],
),
```

Затем маршруты пунктов назначения входят каждый в свой `StatefulShellBranch`, под `StatefulShellRoute` — а login остаётся сверху, вне shell-а:

```
// lib/router.dart — финальная карта: login снаружи, разделы в shell-е
final router = GoRouter(
  initialLocation: '/login',
  routes: [
    GoRoute(path: '/login', // ВНЕ shell-а: login без вкладок
        builder: (context, state) => const LoginScreen()),
    StatefulShellRoute.indexedStack(
      builder: (context, state, navigationShell) =>
          AppShell(navigationShell: navigationShell),
      branches: [
        StatefulShellBranch(routes: [
          // маршрут '/records' со своими под-маршрутами — блок выше
        ]),
        StatefulShellBranch(routes: [
          GoRoute(path: '/wishlist',
              builder: (context, state) => const WishlistScreen()),
        ]),
        // ... и так же: третий StatefulShellBranch, с '/profile'
      ],
    ),
  ],
);
```

Шаг 4 — restart и проверка по четырём пунктам. Карта изменилась → `R`. Отметьте на эмуляторе: (a) вкладки переключаются, и при возвращении каждая — **ровно там, где вы её оставили** (прокрутите список, уйдите, вернитесь); (b) детали открываются **внутри**

вкладки, с видимой панелью и работающим «назад»; (c) у login панели нет; (d) после login `context.go('/records')` приземляется в первой вкладке. Лишь затем: commit.

7. Задание 5 — Формы темы: функциональные локально

Все формы-макеты из L2 становятся функциональными локально — «новая сессия», «оформление заказа», «новая книга»… Рецепт, на типовом примере «Новая пластинка» (экран `StatefulWidget`, как на C6):

```
// screens/record_form_screen.dart — состояние формы
final _formKey = GlobalKey<FormState>();
final _titleCtrl = TextEditingController();
final _priceCtrl = TextEditingController();

@override
void dispose() {
  _titleCtrl.dispose(); // кто создаёт, тот освобождает (C6)
  _priceCtrl.dispose();
  super.dispose();
}
```

Submit валидирует, объявляет и уходит:

```
void _save() {
  if (!_formKey.currentState!.validate()) return; // ошибки уже на экране
  ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Сохранено: ${_titleCtrl.text.trim()}')));
  context.pop(); // открыто push-ем → закрывается pop-ом
}
```

А в `build` поля стоят под `Form(key: _formKey, child: ListView(children: [...]))`. Текстовое поле — validator обязательности:

```
TextFormField(
  controller: _titleCtrl,
  textInputAction: TextInputAction.next,
  decoration: const InputDecoration(labelText: 'Название'),
  validator: (value) => (value == null || value.trim().isEmpty)
      ? 'Название обязательно'
      : null, // null = ВАЛИДНО (идиома C6)
),
```

Числовое поле — подходящая клавиатура плюс правило домена; в конце — кнопка, вызывающая `_save`:

```
TextFormField(
  controller: _priceCtrl,
  keyboardType: const TextInputType.numberWithOptions(decimal: true),
  decoration: const InputDecoration(labelText: 'Цена, MDL'),
  validator: (value) {
    final price = double.tryParse(value ?? '');
    if (price == null) return 'Введите число';
    if (price <= 0) return 'Цена должна быть положительной';
    return null;
  },
),
FilledButton(onPressed: _save, child: const Text('Сохранить')),
```

Требования, на каждую форму:

1. **`Form` + `TextFormField` + `validator`** на каждом поле с правилами; ошибка рисуется сама, красным, под полем.
2. **Контроллеры живут в `State` и умирают в `dispose`** — по одному `dispose()` на контроллер. (Альтернатива `onSaved` + `save()` из C6 принимается — валидация должна существовать, механизм мы не навязываем.)
3. **Подходящие клавиатура и фильтры**: `keyboardType` (`emailAddress`, `phone`, `number`); `inputFormatters` там, где полю нельзя принимать что угодно (`FilteringTextInputFormatter.digitsOnly` на телефоне); `obscureText` на паролях; `textInputAction` `.next`/`.done`.
4. **Специфические валидации темы**, из пункта L3 ТЗ — каждая это `validator` в 3–5 строк; поля выбора валидируются так же: у `DropdownButtonFormField` тоже есть `validator`.
5. **При submit**: `validate()` → невалидно: `return`; валидно: `SnackBar` подтверждения + `pop()`/`go()` по смыслу. Данные **никуда не сохраняются** — список не растёт: это контракт L3, состояние приходит на L4. Говорите это на защите спокойно — это не дефект.

8. Задание 6 — Аутентификация: локальная валидация, `go` после входа

Запрет из L2 («без `Form`, без `validator:`») истёк на C6. Login становится функциональным **локально**, по контракту из каталога: статический на L2 → **валидированный на L3** → подключённый к backend-у на L5. Конкретно:

- экран становится `StatefulWidget` (контроллеры требуют `State`), с `Form` + `_formKey`;
- **e-mail**: непустой, с правдоподобным форматом (содержит `@`, как на C6; полный regex опционален) и клавиатурой `emailAddress`;
- **пароль**: минимум 6 символов, с `obscureText: true` (он у вас уже есть из L2);
- при успешном submit — **`context.go`**, не `push`:

```dart
// login_screen.dart — кнопка, которая валидирует и уходит
FilledButton(
  onPressed: () {
    if (!_formKey.currentState!.validate()) return;
    context.go('/records'); // go, НЕ push: login УХОДИТ из стека
  },
  child: const Text('Войти'),
),
```

Почему `go`: с `push` кнопка «назад» вернула бы уже вошедшего пользователя в login — классическая ошибка, явно проверяемая на защите.

Без реальной аутентификации: любые e-mail и пароль, проходящие валидацию, входят — не сравнивайте с зашитыми в код значениями, не имитируйте пользователей. Проверка по реальному пользователю — материал L5 (backend + JWT), запоминание состояния аутентификации — L4+. На L3 валидация и **есть** контракт.

9. Задание 7 — Feedback: `SnackBar` и диалог подтверждения

Правило C6: только объявляешь — `SnackBar`; просишь решение — диалог с `await`. В приложении должны появиться оба: `SnackBar` при действиях без другого видимого эффекта сегодня («Добавлено в корзину», «Сохранено» — два у вас уже есть из Заданий 5–6) и **диалог подтверждения при деструктивном действии**. Типовой пример — «Выйти из аккаунта» в профиле, полностью функциональный на L3:

```dart
// profile_screen.dart — блокирующее решение → AlertDialog с результатом (C6)
Future<void> _logout(BuildContext context) async {
  final ok = await showDialog<bool>(
    context: context,
    builder: (dialogCtx) => AlertDialog(
      title: const Text('Выйти из аккаунта?'),
      actions: [
        // + TextButton «Нет» → Navigator.pop(dialogCtx, false)
        FilledButton(
            onPressed: () => Navigator.pop(dialogCtx, true),
            child: const Text('Да, выйти')),
      ],
    ),
  );
  if (ok == true && context.mounted) {
    context.go('/login'); // go: раздел с вкладками уходит из стека
  }
}
```

Три детали, отмечаемые на защите, все из C6: диалог закрывается через **`dialogCtx`** — под вложенными навигаторами shell-а `pop` на context-е экрана закрыл бы что-то другое; результат обрабатывается через **`ok == true`**, а не `ok!` — диалог, закрытый мимо, даёт `null`; после `await` context переиспользуется только под **`context.mounted`**. Если в теме есть

удаление/отмена, диалог ставится на неё: подтверждение + `SnackBar` — контракт L3, реальный эффект приходит на L4.

10. Требования к отчёту и защита

Отчёт сдаётся в формате **PDF** с именем файла `PAM_L3_ИмяСтудента.pdf` и содержит:

1. **Стандартный титульный лист** + тема проекта (номер и название);
2. **Карта маршрутов**: `router.dart` + по фразе на маршрут (какой экран открывает; где `push`, где `go` — и почему);
3. **Цепочка темы и вкладки**: скриншоты потока список → детали (разные id) + скриншот shell-а на каждом пункте назначения;
4. **Формы**: скриншоты с **видимыми ошибками валидации** + таблица правил (поле → правило → сообщение → клавиатура/фильтр);
5. **Аутентификация**: скриншот неудавшейся валидации + фраза о `go`;
6. **Feedback**: скриншоты со `SnackBar`-ом и диалогом подтверждения;
7. **История Git**: `git log --oneline` полностью, от L2 до сегодня.

Защита **индивидуальная**, с приложением на эмуляторе или на физическом телефоне. Продемонстрируйте **полный поток, вживую**: запуск → неверный login (ошибки появляются) → верный login → «назад» НЕ ведёт в login → вкладки переключаются и держат своё состояние → список → детали → назад → форма сначала с ошибками, затем верно → диалог подтверждения. Затем **объясните `router.dart` маршрут за маршрутом** и покажите `git log`. Экран, код которого не может быть объяснён, не засчитывается. Оценка за L3 входит в среднюю по лабораторным, присутствующую в двух компонентах семестровой оценки (текущее оценивание и индивидуальная работа).

11. Критерии оценивания

Компонент	Баллы
Все экраны ТЗ как маршруты; `MaterialApp.router`; модели с `id`	2 б
Цепочка список → детали: `/:id`, `push`, поиск в экране (без `extra`)	1 б
Постоянные вкладки: `StatefulShellRoute` + `NavigationBar`, login снаружи	1 б
Формы из ТЗ: `validator`, контроллеры, освобождённые в `dispose` (или `onSaved`), клавиатуры/фильтры, валидации темы	2 б

Компонент	Баллы
Аутентификация с локальной валидацией + `go` после входа (без дороги назад)	1 б
Feedback: `SnackBar` + диалог подтверждения (`dialogCtx`, обработанный результат)	1 б
Непрерывность + Git: продолжен проект L2, commit на шаг, первый — `pub add`	1 б
Защита: живой поток, `router.dart` маршрут за маршрутом, `git log`, вопросы	1 б
Итого	**10 б**

Компонент вкладок может быть покрыт и эквивалентной навигацией по разделам, одобренной преподавателем. Типичные снижения: новый проект — теряется компонент Git; `push` после login — половина компонента аутентификации; контроллеры без `dispose` или валидация «для вида» — пропорционально; оставшиеся леса `Navigator.push` — половина карты маршрутов.

Примеры контрольных вопросов:

1. `context.go` vs `context.push`: что происходит со стеком? Почему после login — `go`, а из списка к деталям — `push`?
2. Как `7` из `/records/7` доходит до экрана деталей и какой у него тип? Почему путешествует id, а не объект через `extra`?
3. Что возвращает `validator` и что означает `null`? Кто рисует ошибку?
4. Кто создаёт `TextEditingController`, где он живёт и кто его освобождает? Что происходит без `dispose`?
5. Что делает `StatefulShellRoute.indexedStack`, почему login стоит вне shell-а и что сохраняется при переключении между вкладками?
6. Почему `Navigator.pop(dialogCtx)`, а не `pop(context)`? Почему `ok == true`?